

SPLITTING THE DIFFERENCE ON ADVERSARIAL TRAINING (USENIX SEC'24)

(VIRTUAL BY CHE-YU CHAN AND HAO-RONG HUANG)

AI 539: TRUSTWORTHY ML

PART III: POISONING ATTACKS ON NN

Sanghyun Hong

sanghyun.hong@oregonstate.edu



Oregon State
University



TRUE AI
Trustworthy and Responsible AI

POISONING ATTACK THREAT MODELS

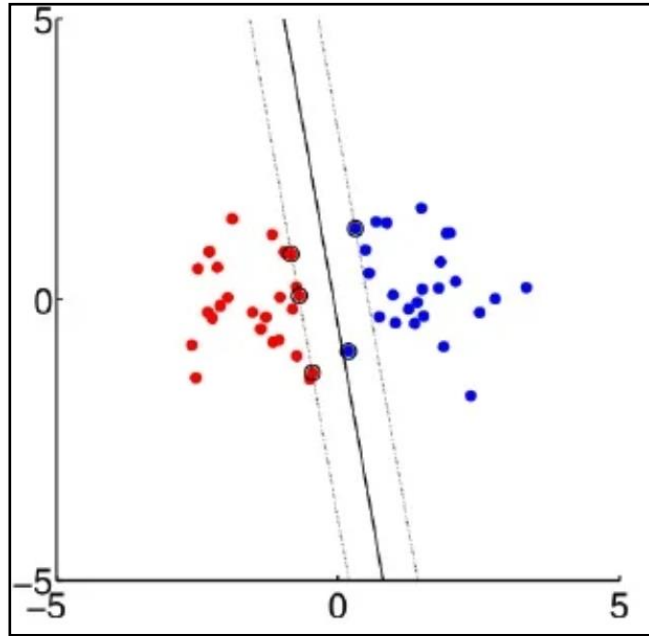
POISONING THREAT MODEL

- Goal
 - Manipulate a ML model's behavior by **compromising the training data**
 - Harm the **integrity** of the training data
- Capability
 - Perturb a subset of samples (D_p) in the training data
 - Inject a few malicious samples (D_p) into the training data
- Knowledge
 - D_{train} : training data
 - D_{test} : test-set data
 - f : a model architecture and its parameters θ
 - A : training algorithm (*e.g.*, SGD)

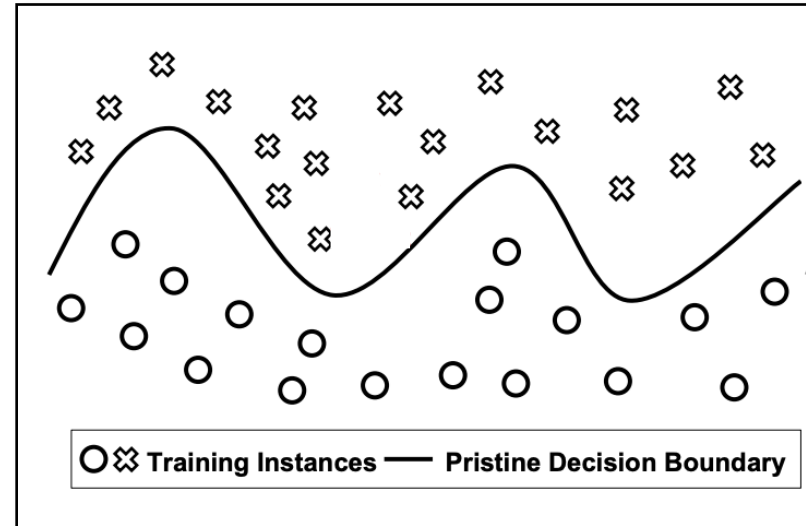
POISONING THREAT MODEL: GOALS

- Goal
 - Manipulate a ML model's behavior by **contaminating the training data**
 - Harm the **integrity** of the training data
- Two well-studied objectives
 - Indiscriminate attack: I want to degrade a model's accuracy
 - Targeted attack: I want misclassification of a specific test-time data

CONCEPTUAL ANALYSIS OF THE POISONING VULNERABILITY

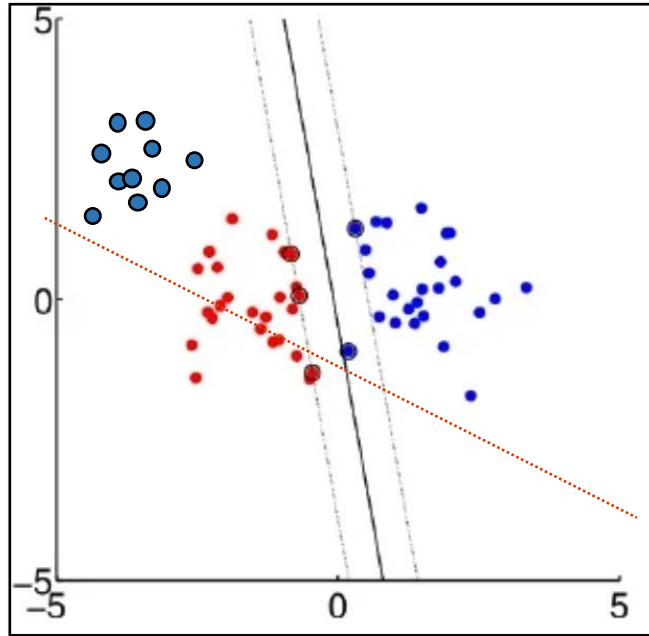


← Linear model (SVM)



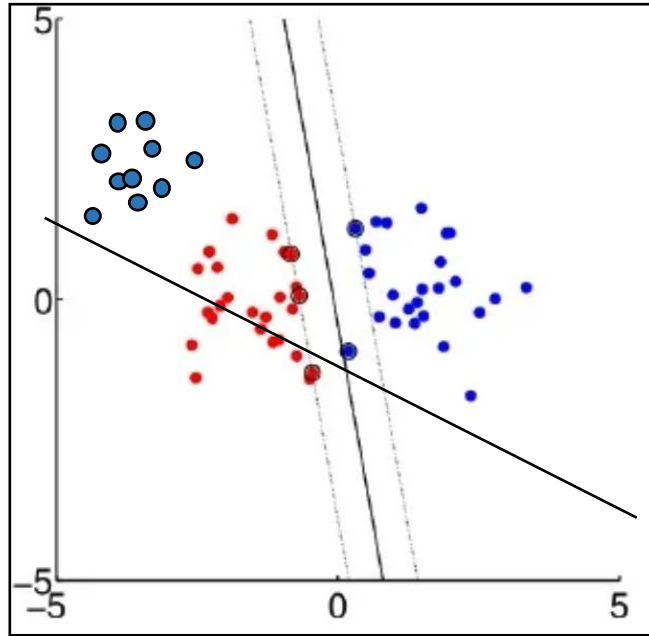
Neural Network →

CONCEPTUAL ANALYSIS OF THE POISONING VULNERABILITY

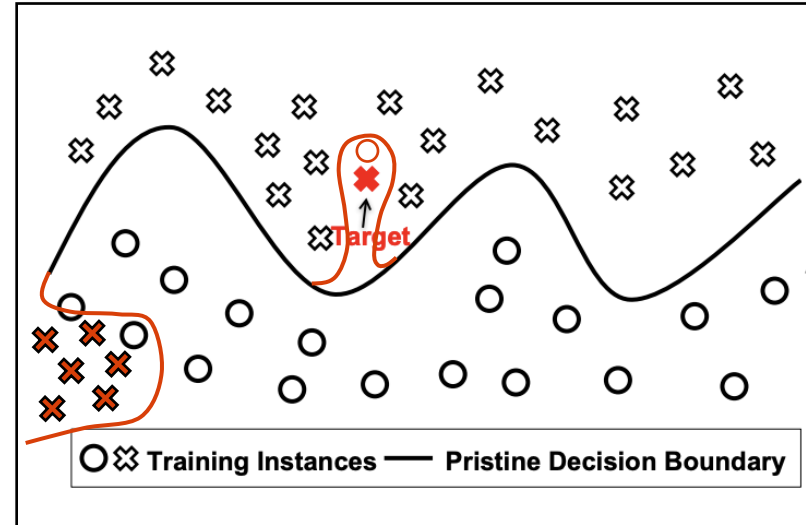


← Linear model (SVM)

CONCEPTUAL ILLUSTRATION OF THE VULNERABILITY TO POISONING



← Linear model (SVM)



Neural Network →

HOW VULNERABLE SVMS ARE?

POISONING ATTACKS AGAINST SUPPORT VECTOR MACHINES, ICML 2012

PRELIMINARIES: SUPPORT VECTOR MACHINE

- DIT [[Link](#)]
 - 1: let's put green points
 - 2: let's put red points on the other side
 - 3: let's put red points closer to the green cluster
 - 4: let's put red points in the middle of the green cluster
 - 5: let's use another kernel.

POISONING THREAT MODEL

- Goal
 - Manipulate a ML model's **accuracy** by compromising the training data
 - In short: **indiscriminate** attack
- Capability
 - Pick a set of test-time samples and craft poisons (x_c, y_c)
 - Inject them into the training data
- Knowledge
 - D_{tr} : training data
 - D_{test} : test-set data (validation data)
 - f : a linear SVM and its parameters θ
 - A : training algorithm (*e.g.*, Sub-gradient descent)

PROPOSED ATTACK ON SUPPORT VECTOR MACHINE

- Indiscriminate attack procedure
 - Draw a set of poison candidates from the validation data
 - Craft poisoning samples
 - Inject them into the original training data
 - Increase the loss of the model trained on the compromised data

PROPOSED ATTACK ON SUPPORT VECTOR MACHINE

Algorithm 1 Poisoning attack against SVM

Input: $\mathcal{D}_{\text{tr}}$, the training data; $\mathcal{D}_{\text{val}}$, the validation data; y_c , the class label of the attack point; $x_c^{(0)}$, the initial attack point; t , the step size.

Output: x_c , the final attack point.

- 1: $\{\alpha_i, b\} \leftarrow$ learn an SVM on $\mathcal{D}_{\text{tr}}$. // train an SVM on the clean data
 - 2: $k \leftarrow 0$.
 - 3: **repeat**
 - 4: Re-compute the SVM solution on $\mathcal{D}_{\text{tr}} \cup \{x_c^{(p)}, y_c\}$ using incremental SVM (e.g., [Cauwenberghs & Poggio, 2001](#)). This step requires $\{\alpha_i, b\}$. // train an SVM with the poison
 - 5: Compute $\frac{\partial L}{\partial u}$ on $\mathcal{D}_{\text{val}}$ according to Eq. (10). // compute the gradient
 - 6: Set u to a unit vector aligned with $\frac{\partial L}{\partial u}$.
 - 7: $k \leftarrow k + 1$ and $x_c^{(p)} \leftarrow x_c^{(p-1)} + tu$ // update the poison, to increase the loss
 - 8: **until** $L(x_c^{(p)}) - L(x_c^{(p-1)}) < \epsilon$ // stop if the loss doesn't increase more than ϵ
 - 9: **return:** $x_c = x_c^{(p)}$
-

PROPOSED ATTACK ON SUPPORT VECTOR MACHINE

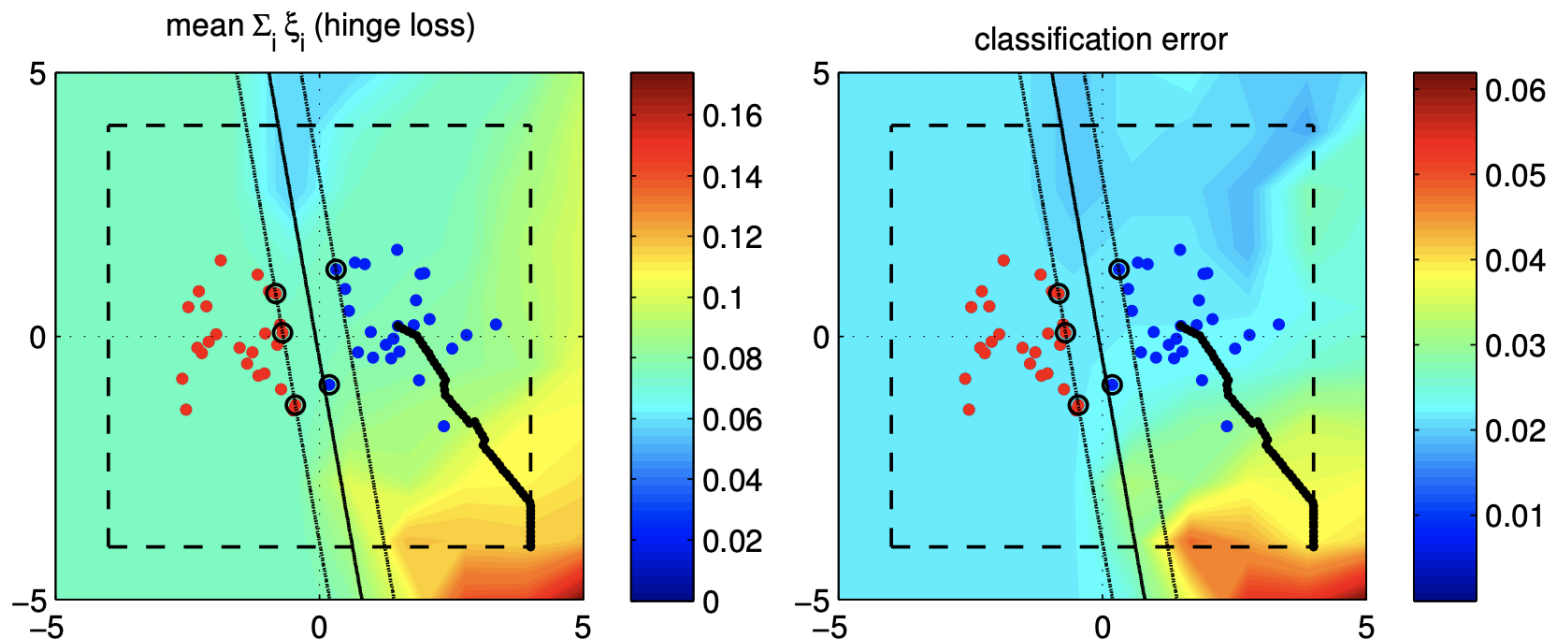
- Indiscriminate attack procedure
 - Draw a set of poison candidates from the validation data
 - Craft poisoning samples
 - **Inject** them into the original training data
 - Increase the loss of the model trained on the compromised data

EVALUATION

- Setup
 - Datasets
 - Artificial data:
 - Binary classification: Gaussian dist. [$N(-1.5, 0.6^2)$ and $N(1.5, 0.6^2)$]
 - Training data : 50 samples, 25 per class
 - Validation data: 1k samples, 500 per class
 - Real data: MNIST
 - Model(s)
 - SVM [Linear vs. RBF-Kernel]

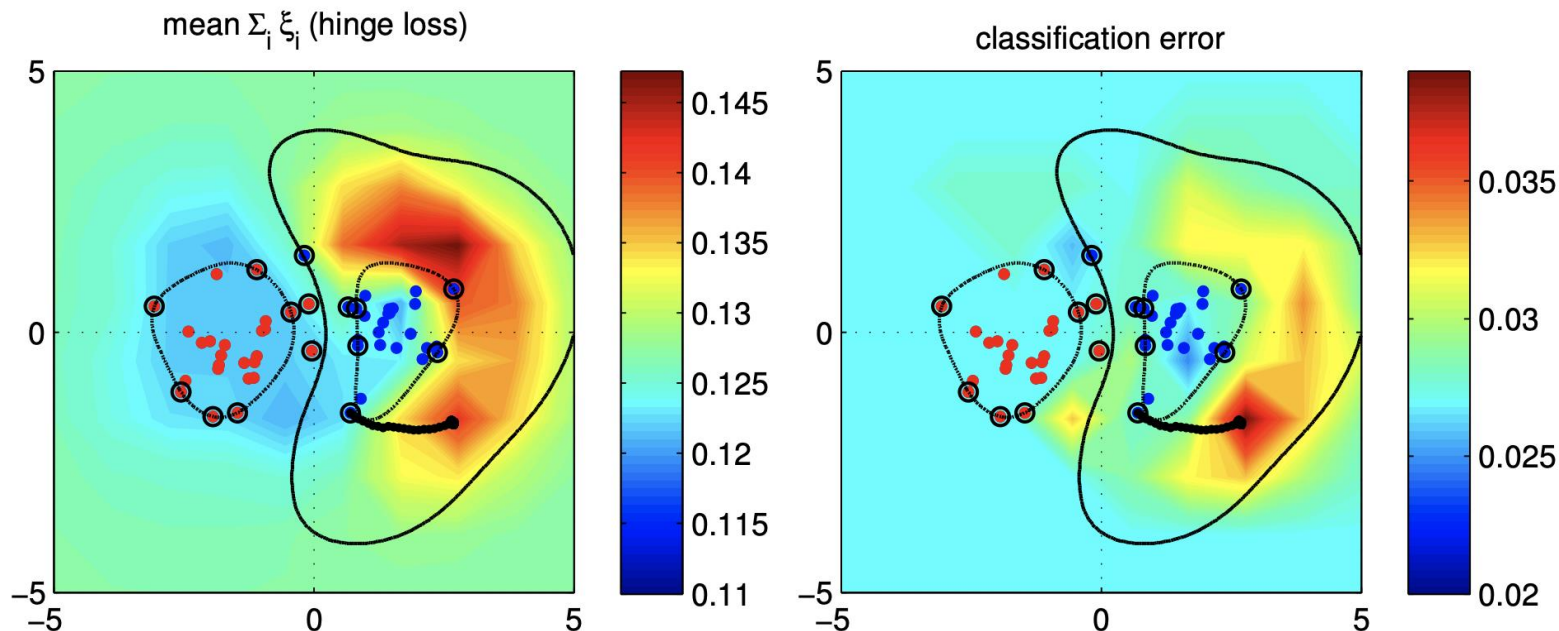
EVALUATION: POISON CRAFTING IN ARTIFICIAL DATA

- Linear SVM



EVALUATION: POISON CRAFTING IN ARTIFICIAL DATA

- SVM with RBF Kernel

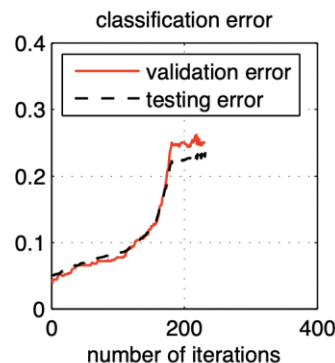
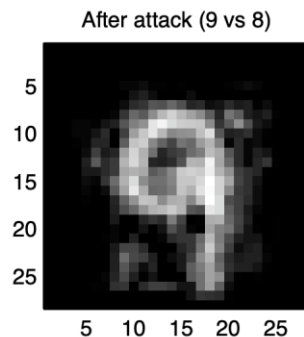
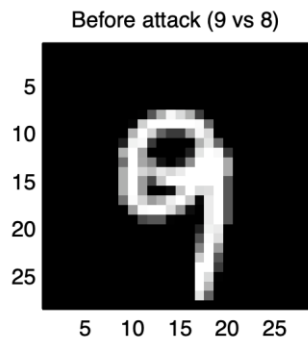
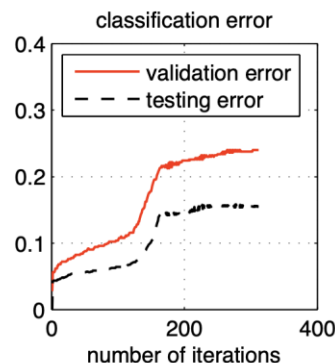
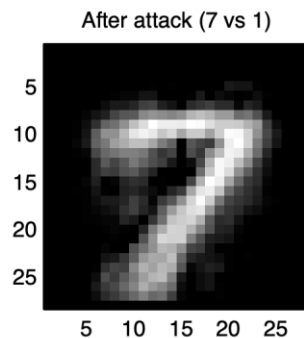
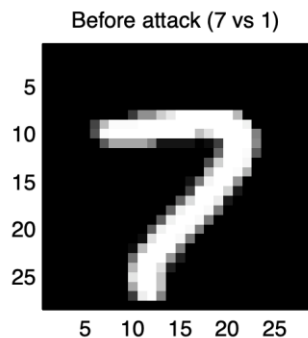


EVALUATION

- Setup
 - Datasets
 - Artificial data:
 - Binary classification: Gaussian dist. [$N(-1.5, 0.6^2)$ and $N(1.5, 0.6^2)$]
 - Training data : 50 samples, 25 per class
 - Validation data: 1k samples, 500 per class
 - Real data: MNIST
 - 7 vs 1 | 9 vs 8 | 4 vs 0
 - Training data : 200 samples, 100 per class
 - Validation data: 1k samples, 500 per class
 - Testing data : 4k samples, 2k per class
 - Model(s)
 - SVM [Linear vs. RBF-Kernel]

EVALUATION: REAL-DATA (MNIST)

- Linear SVM

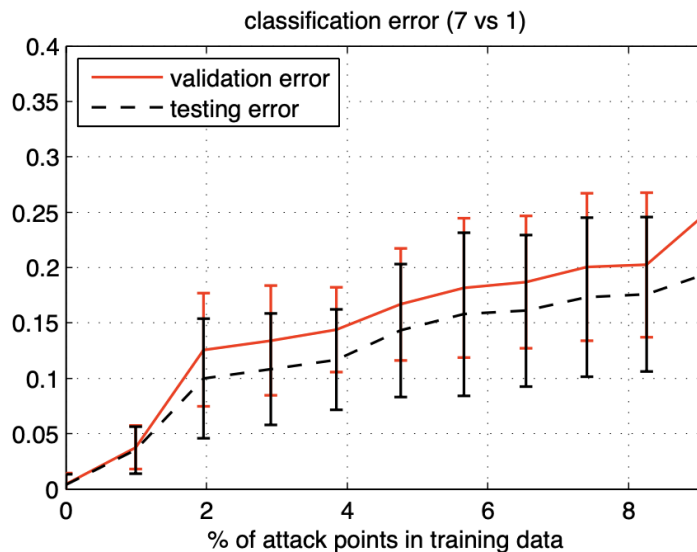


- Results

- Use a *single* poison
- Error increases by 15 – 20%

EVALUATION: REAL-DATA (MNIST)

- Linear SVM



- Results

- Use a *single* poison
- Error increases by 15 – 20%
- Increasing # poisons leads to a higher error

HOW VULNERABLE REGRESSION MODELS ARE?

MANIPULATING MACHINE LEARNING: POISONING ATTACKS AND COUNTERMEASURES FOR REGRESSION LEARNING, IEEE S&P 2018

BACKGROUND: REGRESSION MODELS

- Regression Models [[Demo](#)]
 - DIT
 - 1. let's add some more points
 - 2. let's see how much error ($RMSE$) it increases
 - In the Paper
 - Ordinary Least Squares (OLS)
 - Ridge regression
 - LASSO
 - Elastic-net regression

THREAT MODEL

- Goal
 - Indiscriminate attack (increase the error on D_{val})
- Capability
 - Train a model f on D_{tr}
 - Inject p poisons into the training set ($N(D_{tr}) = n + p$)
- Knowledge [White-box vs. Black-box]
 - D_{tr} : training data (black-box adversary only has partial knowledge of D_{tr})
 - D_{val} : validation data
 - f : a model and its parameters (black-box attacker doesn't know the parameters)
 - L : training algorithm

POISONING AS A BI-LEVEL OPTIMIZATION

$$\begin{aligned} \arg \max_{\mathcal{D}_p} \quad & \mathcal{W}(\mathcal{D}', \boldsymbol{\theta}_p^*), \\ \text{s.t.} \quad & \boldsymbol{\theta}_p^* \in \arg \min_{\boldsymbol{\theta}} \mathcal{L}(\mathcal{D}_{\text{tr}} \cup \mathcal{D}_p, \boldsymbol{\theta}) \end{aligned}$$

- Outer-optimization: maximize the error of a model on the validation data
- Inner-optimization: minimize the model's error on the training data

PROPOSED POISONING ATTACK ON REGRESSION MODELS

Algorithm 1 Poisoning Attack Algorithm

Input: $\mathcal{D} = \mathcal{D}_{\text{tr}}$ (white-box) or $\mathcal{D}'_{\text{tr}}$ (black-box), $\mathcal{D}'$, $\mathcal{L}$, $\mathcal{W}$, the initial poisoning attack samples $\mathcal{D}_p^{(0)} = (\mathbf{x}_c, y_c)_{c=1}^p$, a small positive constant ε .

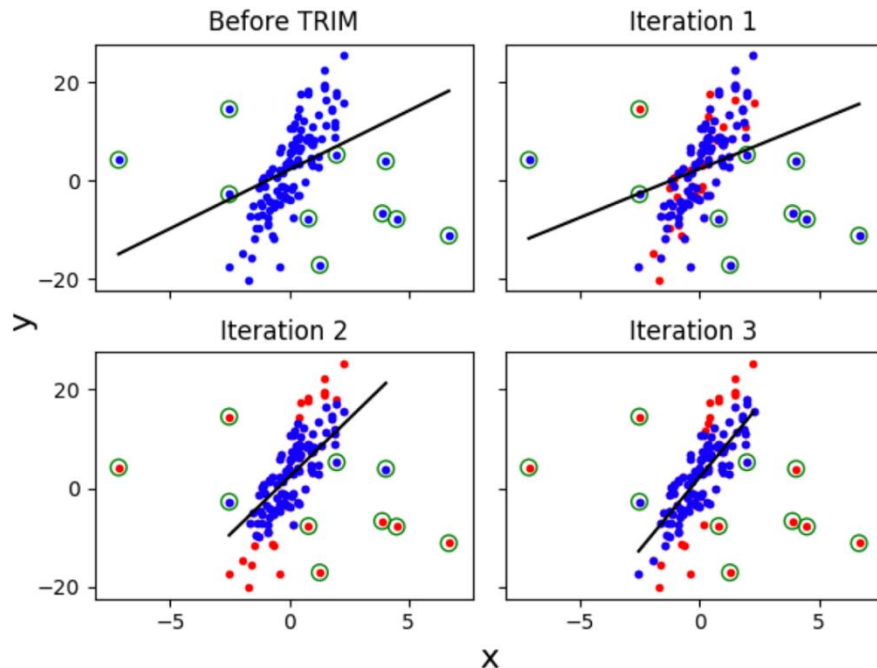
```
1:  $i \leftarrow 0$  (iteration counter)
2:  $\boldsymbol{\theta}^{(i)} \leftarrow \arg \min_{\boldsymbol{\theta}} \mathcal{L}(\mathcal{D} \cup \mathcal{D}_p^{(i)}, \boldsymbol{\theta})$  // train a model on the contaminated data
3: repeat
4:    $w^{(i)} \leftarrow \mathcal{W}(\mathcal{D}', \boldsymbol{\theta}^{(i)})$ 
5:    $\boldsymbol{\theta}^{(i+1)} \leftarrow \boldsymbol{\theta}^{(i)}$ 
6:   for  $c = 1, \dots, p$  do
7:      $\mathbf{x}_c^{(i+1)} \leftarrow \text{line\_search}(\mathbf{x}_c^{(i)}, \nabla_{\mathbf{x}_c} \mathcal{W}(\mathcal{D}', \boldsymbol{\theta}^{(i+1)}))$  // update poisons to increase the loss of the model
8:      $\boldsymbol{\theta}^{(i+1)} \leftarrow \arg \min_{\boldsymbol{\theta}} \mathcal{L}(\mathcal{D} \cup \mathcal{D}_p^{(i+1)}, \boldsymbol{\theta})$ 
9:      $w^{(i+1)} \leftarrow \mathcal{W}(\mathcal{D}', \boldsymbol{\theta}^{(i+1)})$ 
10:   $i \leftarrow i + 1$ 
11: until  $|w^{(i)} - w^{(i-1)}| < \varepsilon$  // stop when the model doesn't change more than  $\varepsilon$ 

Output: the final poisoning attack samples  $\mathcal{D}_p \leftarrow \mathcal{D}_p^{(i)}$ 
```

PROPOSED DEFENSE: TRIM

Algorithm 2 [TRIM algorithm]

- 1: **Input:** Training data $\mathcal{D} = \mathcal{D}_{\text{tr}} \cup \mathcal{D}_p$ with $|\mathcal{D}| = N$;
number of attack points $p = \alpha \cdot n$.
- 2: **Output:** θ .
- 3: $\mathcal{I}^{(0)} \leftarrow \{1, \dots, N\}$ /* First train with all samples */
- 4: $\theta^{(0)} \leftarrow \arg \min_{\theta} \mathcal{L}(\mathcal{D}^{\mathcal{I}^{(0)}}, \theta)$ /* Initial estimation of θ */
- 5: $i \leftarrow 0$ /* Iteration count */
- 6: **repeat**
- 7: $i \leftarrow i + 1$;
- 8: $\mathcal{I}^{(i)} \leftarrow$ subset of size n that min. $\mathcal{L}(\mathcal{D}^{\mathcal{I}^{(i)}}, \theta^{(i-1)})$
- 9: $\theta^{(i)} \leftarrow \arg \min_{\theta} \mathcal{L}(\mathcal{D}^{\mathcal{I}^{(i)}}, \theta)$ /* Current estimator */
- 10: $R^{(i)} = \mathcal{L}(\mathcal{D}^{\mathcal{I}^{(i)}}, \theta^{(i)})$ /* Current loss */
- 11: **until** $i > 1 \wedge R^{(i)} = R^{(i-1)}$ /* Convergence condition */
- 12: **return** $\theta^{(i)}$ /* Final estimator */.



EVALUATION

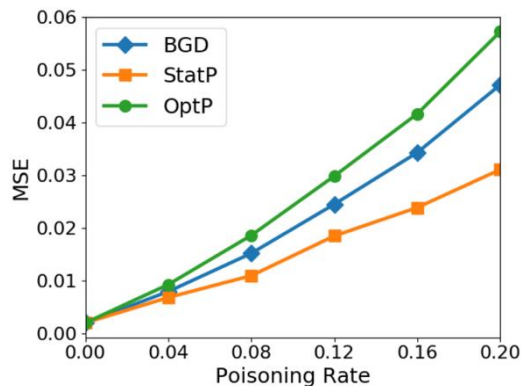
- Setup
 - Datasets: Health care | Loan | Housing
 - Models
 - Ordinary Least Square (OLS)
 - Ridge regression
 - LASSO
 - Elastic-net regression
 - Attacks
 - OptP | StatP | BGD (Prior work by Xiao *et al.*)
 - Defenses
 - Huber | RANSAC | Chen *et al.* | RONI | TRIM

EVALUATION

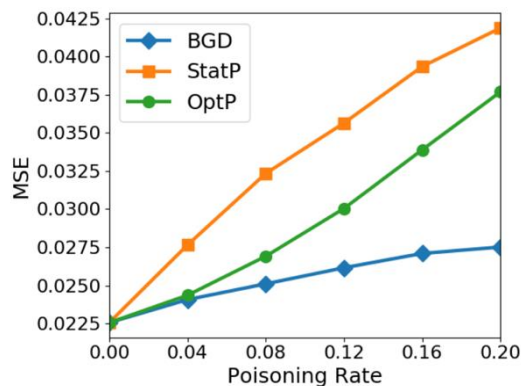
- Results Summary

- Attacks

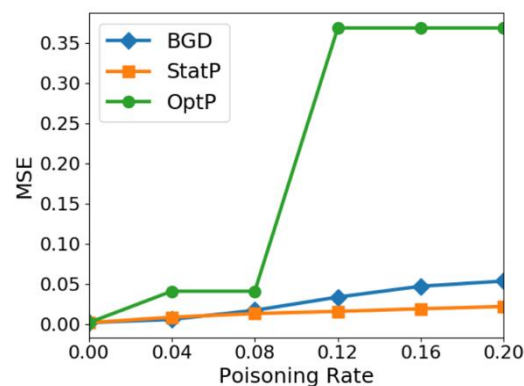
- OptP > StatP, BGD (Prior work)
 - StatP, BGD: varies from datasets
 - StatP > OptP: computational efficiency; StatP still shows a reasonable success rate
 - Poisons *transfer*: crafted on one model works for the three others



(a) Health Care Dataset



(b) Loan Dataset



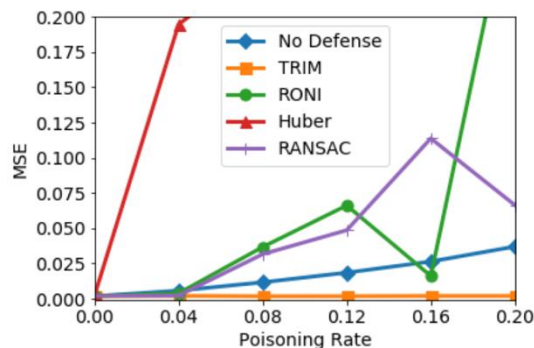
(c) House Price Dataset

EVALUATION

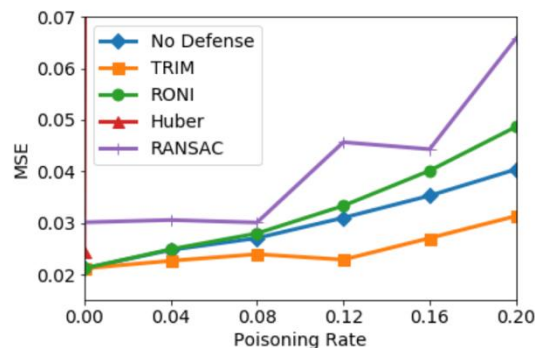
- Results Summary

- Defenses

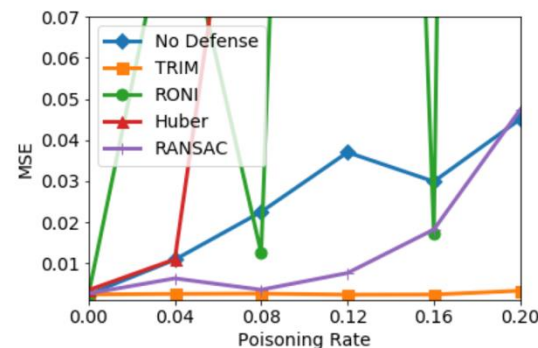
- TRIM > Huber | RANSAC | Chen *et al.* | RONI
 - TRIM is computationally efficient (< 0.02 seconds on the House dataset)
 - Prior work's defenses sometimes increase errors



(a) Health Care Dataset



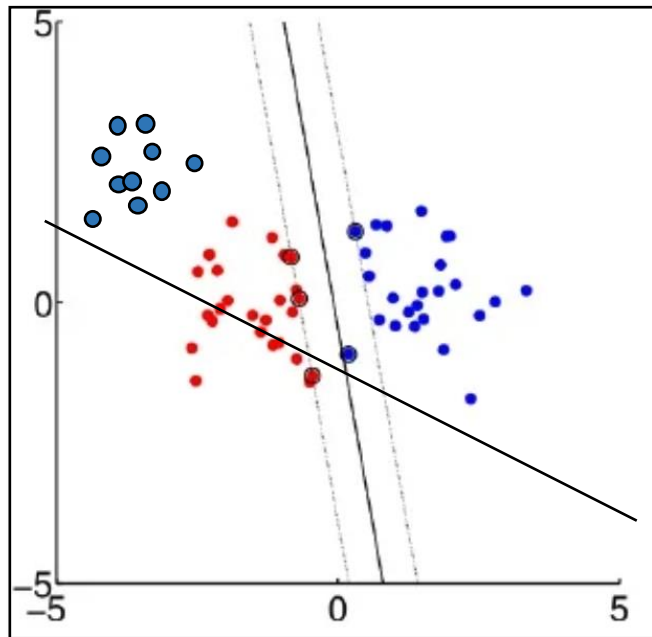
(b) Loan Dataset



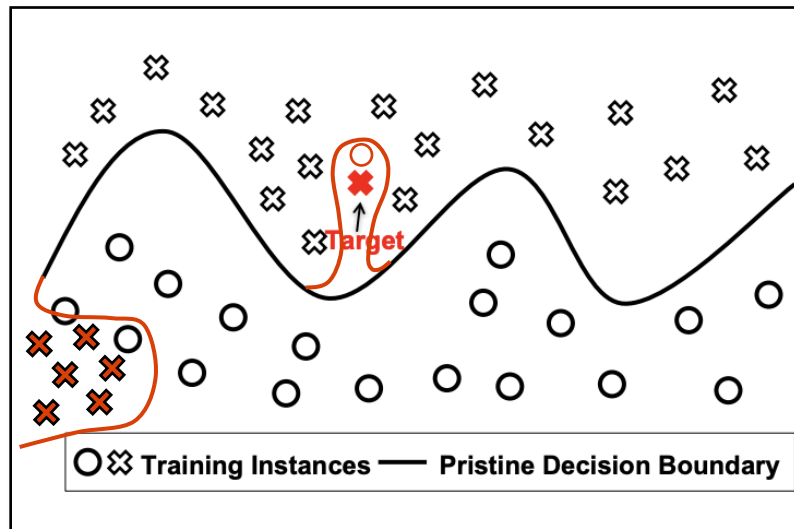
(c) House Price Dataset

TARGETED POISONING ATTACKS

RECAP: CONCEPTUAL ILLUSTRATION OF THE VULNERABILITY TO POISONING



← Linear model (SVM)



Neural Network →

TARGETED POISONING THREAT MODEL

- Goal
 - **Targeted** attack
 - Model causes a misclassification of (x_t, y_t) , while preserving acc. on D_{val}
- Capability
 - Know a target (x_t, y_t)
 - Pick p candidates from test data $(x_{c1}, y_{c1}), (x_{c2}, y_{c2}) \dots$ and craft poisons $(x_{p1}, y_{p1}), (x_{p2}, y_{p2}) \dots$
 - Inject them into the training data
- Knowledge
 - D_{tr} : training data
 - D_{test} : test-set data (validation data)
 - f : a model and its parameters θ
 - A : training algorithm (e.g., mini-batch SGD)

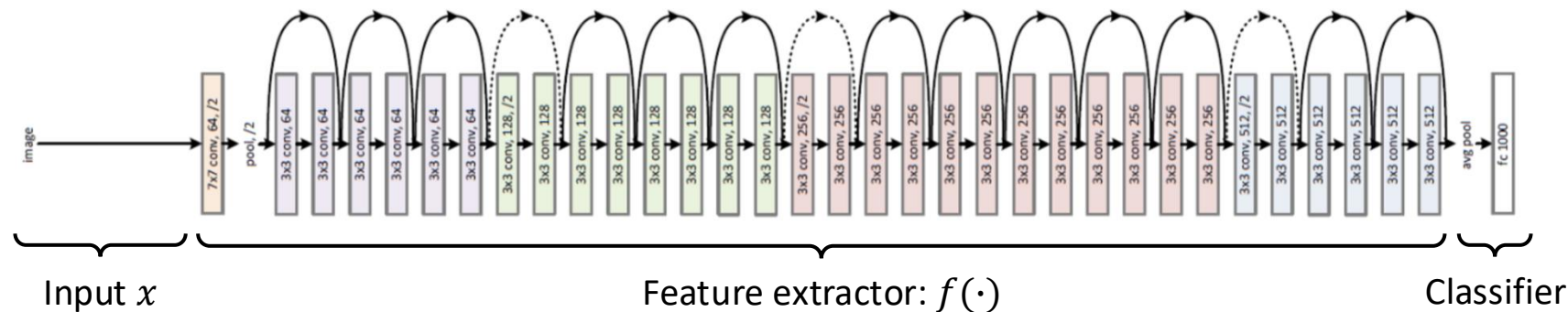
TARGETED POISONING THREAT MODEL

- Goal
 - Targeted **clean-label** ($y_{c1} = y_{p1}$) attack
 - Model causes a misclassification of (x_t, y_t) , while preserving acc. on D_{val}
- Capability
 - Know a target (x_t, y_t)
 - Pick p candidates from test data (x_{c1}, y_{c1}) , (x_{c2}, y_{c2}) ... and craft poisons (x_{p1}, y_{p1}) , (x_{p2}, y_{p2}) ...
 - Inject them into the training data
- Knowledge
 - D_{tr} : training data
 - D_{test} : test-set data (validation data)
 - f : a model and its parameters θ
 - A : training algorithm (e.g., mini-batch SGD)

HOW VULNERABLE NEURAL NETWORKS ARE TO TARGETED ATTACKS?

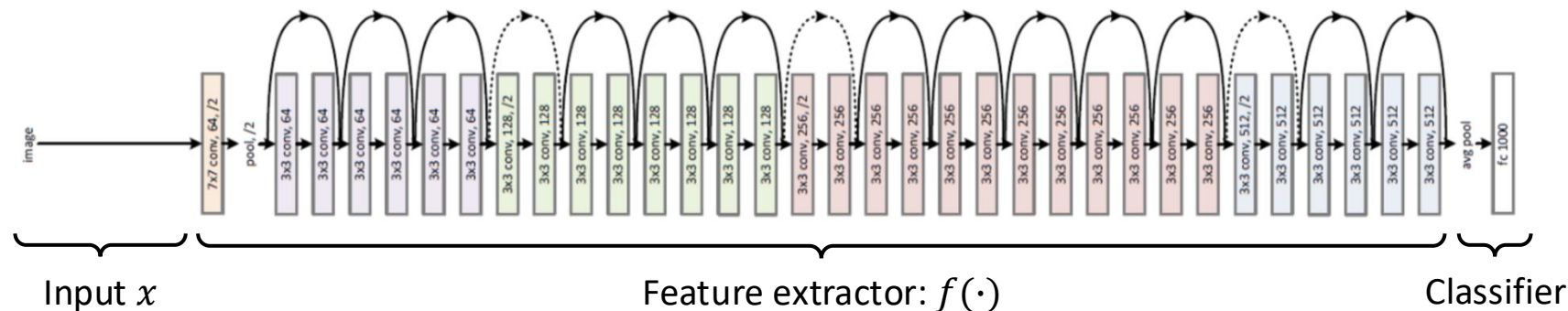
POISON FROGS! TARGETED CLEAN-LABEL POISONING ATTACKS ON NEURAL NETWORKS, SHAFABI ET AL., NEURIPS 2018

BACKGROUND: CONVOLUTIONAL NEURAL NETWORKS



- A conventional view:
 - Convolutions: extract features, embeddings, latent representations, ...
 - Last layer: uses the output for a classification task

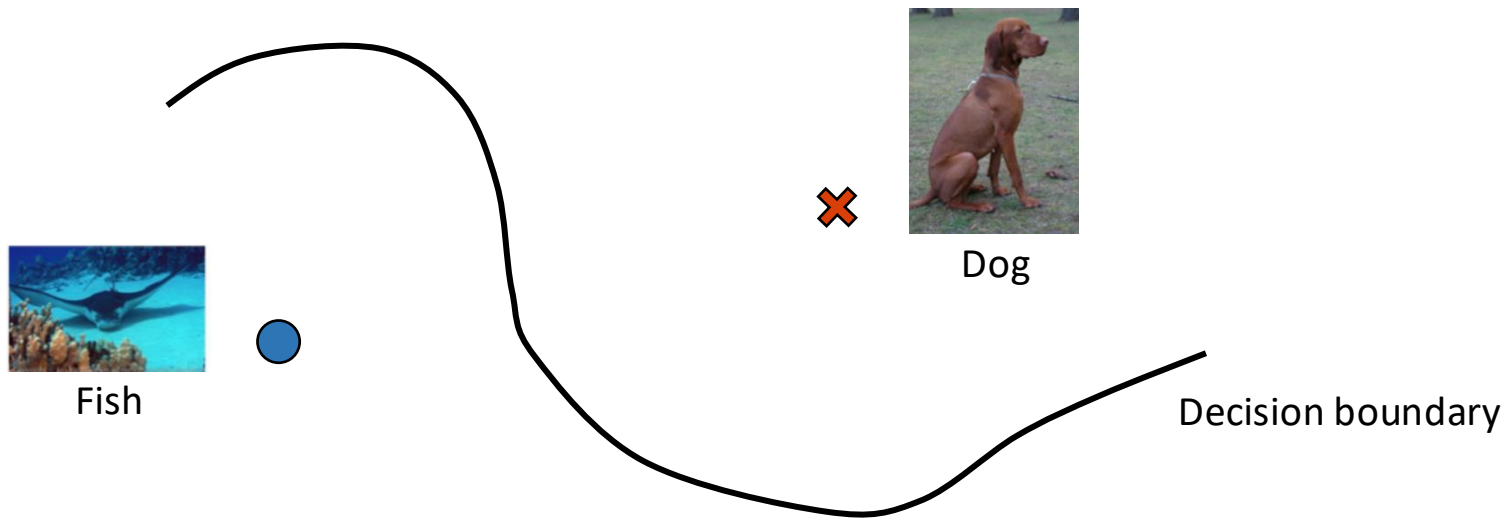
BACKGROUND: CONVOLUTIONAL NEURAL NETWORKS



- Input-space $\neq$ Feature-space:
 - Two samples similar in the input-space can be far from each other in the feature-space
 - Two samples very different in the input-space can be close to each other in f

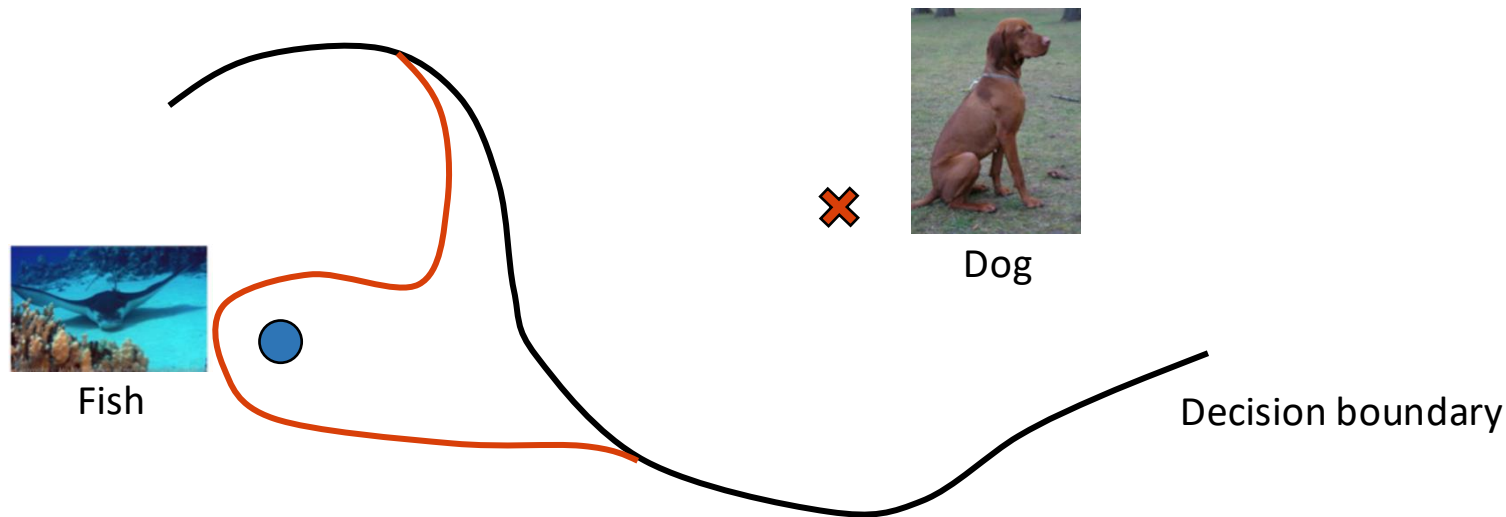
THE KEY IDEA: FEATURE COLLISION

- Goal
 - You want your *any* poison to be closer to your target (x_t, y_t) in the *feature space*



THE KEY IDEA: FEATURE COLLISION

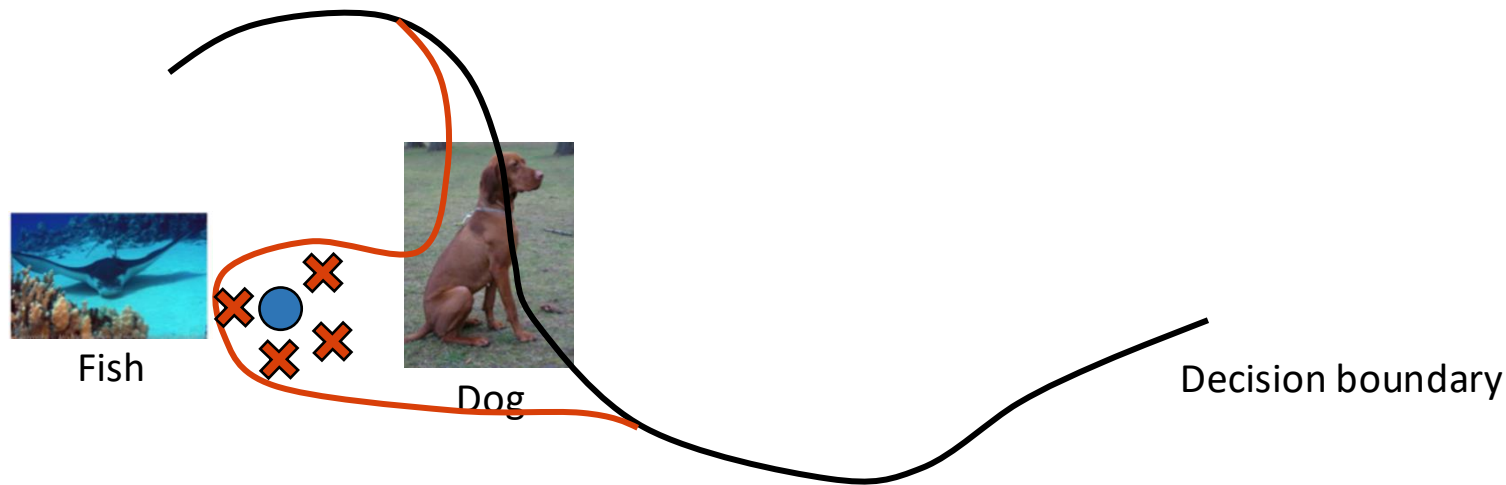
- Goal
 - You want your *any* poison to be closer to your target (x_t, y_t) in the *feature space*



The Fish Becomes **DogFish!**

THE KEY IDEA: FEATURE COLLISION

- Goal
 - You want your *any* poison to be closer to your target (x_t, y_t) in the *feature space*



THE KEY IDEA: FEATURE COLLISION

- Goal

- You want your *any* poison to be closer to your target (x_t, y_t) in the *feature space*
- Objective:

$$\mathbf{p} = \underset{\mathbf{x}}{\operatorname{argmin}} \quad \|f(\mathbf{x}) - f(\mathbf{t})\|_2^2 + \beta \|\mathbf{x} - \mathbf{b}\|_2^2$$

- Optimization:

Algorithm 1 Poisoning Example Generation

Input: target instance t , base instance b , learning rate λ

Initialize $\mathbf{x}$: $x_0 \leftarrow b$

Define: $L_p(x) = \|f(\mathbf{x}) - f(\mathbf{t})\|^2$

for $i = 1$ **to** $maxIters$ **do**

 Forward step: $\hat{x}_i = x_{i-1} - \lambda \nabla_x L_p(x_{i-1})$ // construct input perturbations

 Backward step: $x_i = (\hat{x}_i + \lambda \beta b) / (1 + \beta \lambda)$ // decide how much we will perturb

end for

EVALUATIONS

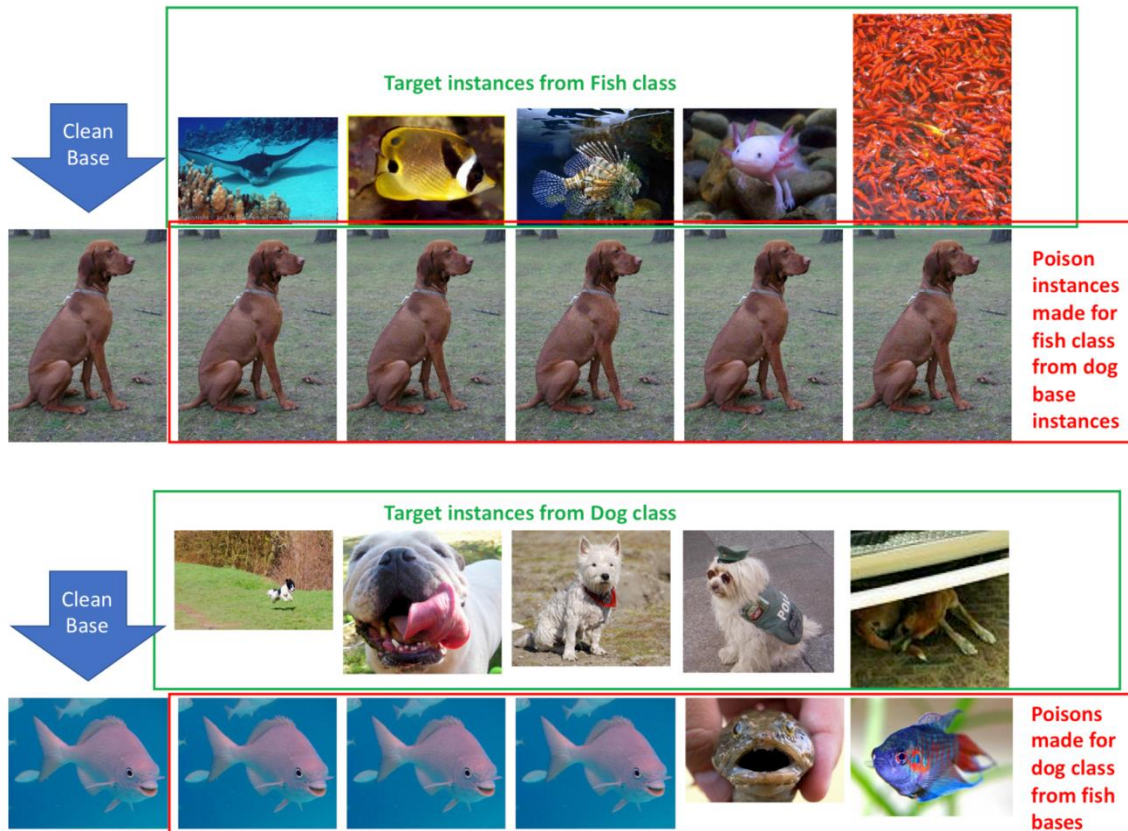
- Scenarios
 - Scenario 1: Transfer learning
 - Scenario 2: End-to-end learning

EVALUATIONS: TRANSFER LEARNING

- Setup
 - Dataset: Dog vs. Fish (ImageNet)
 - Models: Inception-V3 (Pretrained on ImageNet)
- “one-shot kill” Attacks
 - Goal: Dog > Fish or Fish > Dog | All 1099 targets from the test-set
 - Craft a poison using a single image chosen from the other class
 - Train the last layer on $D_{tr} \cup (x_p, y_p)$ and check if the target’s label is flipped
- Results
 - The attack succeeds with 100% accuracy
 - The accuracy drop caused by the attack is 0.2% on average

EVALUATIONS: TRANSFER LEARNING

- Examples

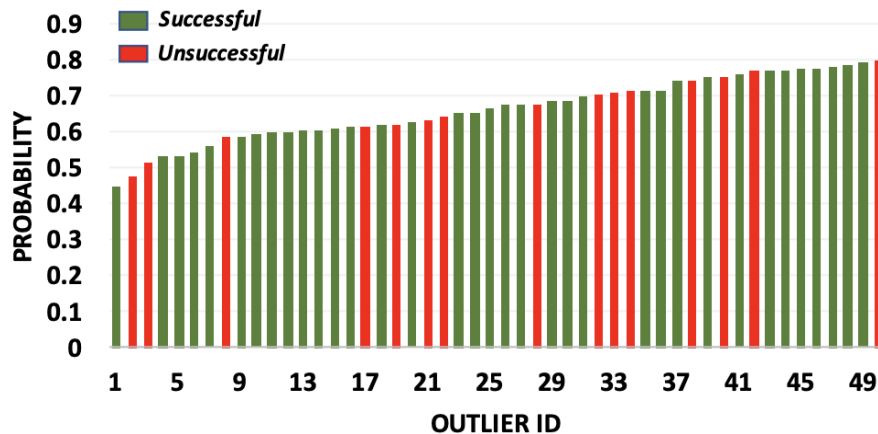
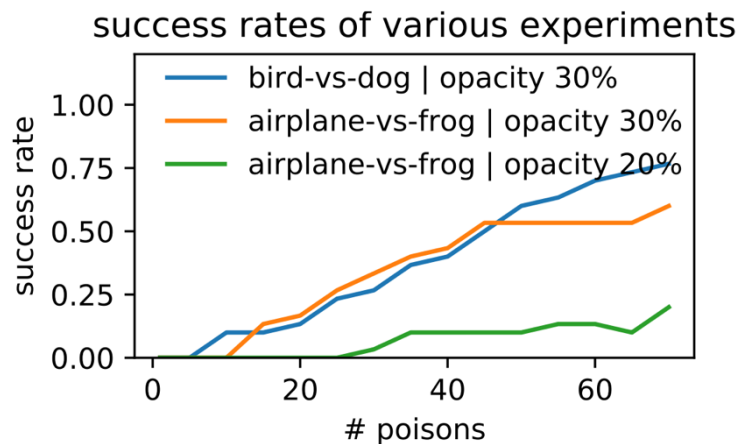


EVALUATIONS: END-TO-END LEARNING

- Setup
 - Dataset: CIFAR-10
 - Models: AlexNet (Pretrained on CIFAR-10)
- “end-to-end” Attacks
 - Goal: Bird > Dog or Airplane > Frog
 - Craft 1-70 poisons using the images chosen from the (Dog or Frog) class
 - Trick: watermarking!
 - Train the entire model on $D_{tr} \cup (x_p, y_p)$ and check the misclassification rate

EVALUATIONS: END-TO-END LEARNING

- Results



CAN WE IMPROVE THE TRANSFERABILITY OF TARGETED ATTACKS?

METAPOISON! PRACTICAL GENERAL-PURPOSE CLEAN-LABEL DATA POISONING, HUANG ET AL., NEURIPS 2020

REVISIT: POISONING THREAT MODEL

- Goal
 - Targeted **clean-label** ($y_{c1} = y_{p1}$) attack
 - Model causes a misclassification of (x_t, y_t) , while preserving acc. on D_{val}
- Capability
 - Know a target (x_t, y_t)
 - Pick p candidates from test data $(x_{c1}, y_{c1}), (x_{c2}, y_{c2}) \dots$ and craft poisons $(x_{p1}, y_{p1}), (x_{p2}, y_{p2}) \dots$
 - Inject them into the training data
- Knowledge
 - D_{tr} : training data
 - D_{test} : test-set data (validation data)
 - f : a model and its parameters θ
 - A : training algorithm (e.g., mini-batch SGD)

REVISIT: THE KEY IDEA – FEATURE COLLISION

- Goal

- Your poisons should work against any f and θ
- Objective:

$$\mathbf{p} = \underset{\mathbf{x}}{\operatorname{argmin}} \underbrace{\|f(\mathbf{x}) - f(\mathbf{t})\|_2^2}_{\text{Now you don't know the } f, \text{ how can you estimate this?}} + \beta \|\mathbf{x} - \mathbf{b}\|_2^2$$

- Revisit the previous idea

- Bi-level optimization

$$\begin{aligned} \arg \max_{\mathcal{D}_p} \quad & \mathcal{W}(\mathcal{D}', \boldsymbol{\theta}_p^*), \\ \text{s.t.} \quad & \boldsymbol{\theta}_p^* \in \arg \min_{\boldsymbol{\theta}} \mathcal{L}(\mathcal{D}_{\text{tr}} \cup \mathcal{D}_p, \boldsymbol{\theta}) \end{aligned}$$

$$\begin{aligned} X_p^* &= \underset{X_p}{\operatorname{argmin}} \mathcal{L}_{\text{adv}}(x_t, y_{\text{adv}}; \boldsymbol{\theta}^*(X_p)) \\ \boldsymbol{\theta}^*(X_p) &= \underset{\boldsymbol{\theta}}{\operatorname{argmin}} \mathcal{L}_{\text{train}}(X_c \cup X_p, Y; \boldsymbol{\theta}). \end{aligned}$$

Problem: no control over θ

THE CHALLENGE: LEARNING PROCESS

- Mode parameters are not fixed!
 - Initialization
 - Mini-batch-ed data
 - # of training epochs

Algorithm

Input: Examples $\{x_1, \dots, x_N\}$, loss function $\mathcal{L}(\theta) = \frac{1}{N} \sum_i \mathcal{L}(\theta, x_i)$. Parameters: learning rate η_t , noise scale σ , group size L , gradient norm bound C .

Initialize θ_0 randomly

for $t \in [T]$ **do**

Compute gradient

 For each $i \in L_t$, compute $\mathbf{g}_t(x_i) \leftarrow \nabla_{\theta_t} \mathcal{L}(\theta_t, x_i)$

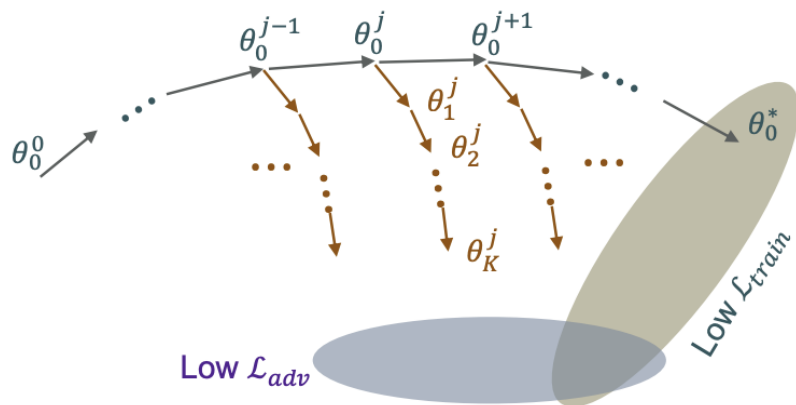
Descent

$\theta_{t+1} \leftarrow \theta_t - \eta_t \tilde{\mathbf{g}}_t$

Output θ_T and compute the overall privacy cost (ϵ, δ) using a privacy accounting method.

THE KEY IDEA: UNROLLING

- Goal
 - You *simulate all* the training procedures with *possible* f, θ s while crafting your poisons



Algorithm 1 Craft poison examples via MetaPoison

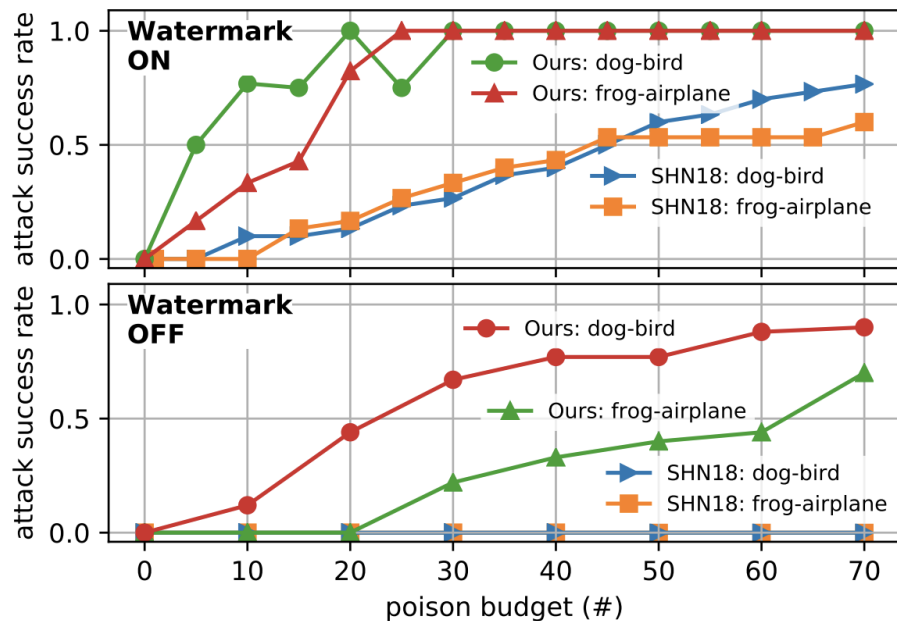
- 1: **Input** Training set of images and labels (X, Y) of size N , target image x_t , adversarial class y_{adv} , ϵ and ϵ_c thresholds, $n \ll N$ subset of images to be poisoned, T range of training epochs, M randomly initialized models.
- 2: **Begin**
- 3: Stagger the M models, training the m th model weights θ_m up to $\lfloor mT/M \rfloor$ epochs
- 4: Select n images from the training set to be poisoned, denoted by X_p . Remaining clean images denoted X_c
- 5: For $i = 1, \dots, C$ crafting steps:
- 6: For $m = 1, \dots, M$ models:
- 7: Copy $\tilde{\theta} = \theta_m$
- 8: For $k = 1, \dots, K$ unroll steps^a:
- 9: $\tilde{\theta} = \tilde{\theta} - \alpha \nabla_{\tilde{\theta}} \mathcal{L}_{train}(X_c \cup X_p, Y; \tilde{\theta})$
- 10: Store adversarial loss $\mathcal{L}_m = \mathcal{L}_{adv}(x_t, y_{adv}; \tilde{\theta})$
- 11: Advance epoch $\theta_m = \theta_m - \alpha \nabla_{\theta_m} \mathcal{L}_{train}(X, Y; \theta_m)$
- 12: If θ_m is at epoch $T + 1$:
- 13: Reset θ_m to epoch 0 and reinitialize
- 14: Average adversarial losses $\mathcal{L}_{adv} = \sum_{m=1}^M \mathcal{L}_m / M$
- 15: Compute $\nabla_{X_p} \mathcal{L}_{adv}$
- 16: Update X_p using Adam and project onto ϵ, ϵ_c ball
- 17: **Return** X_p

EVALUATION

- Setup
 - Dataset: CIFAR-10
 - Models: 6-layer ConveNet (default), ResNet20, VGG13
 - Attack hyper-parameters:
 - C: 60 | M: 24 | K: 2
- Attacks
 - 30 randomly chosen targets
 - Increase the # poisons from 1 – 10% of the training data n
 - Baseline:
 - Poison Frogs!

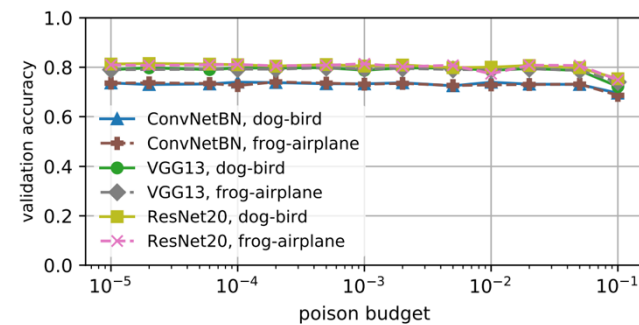
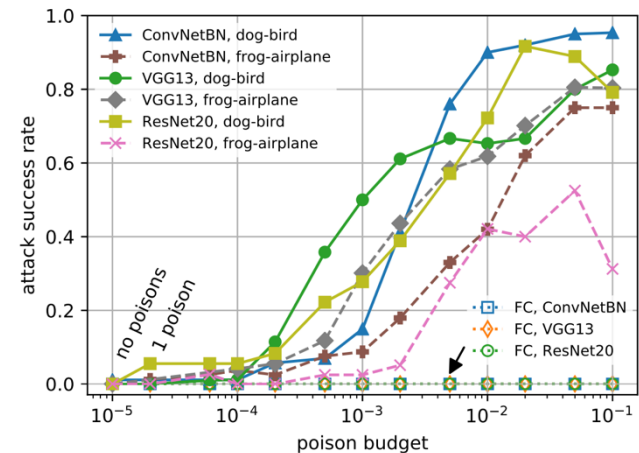
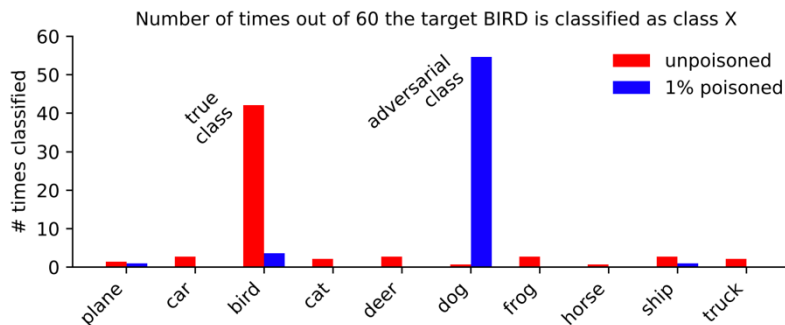
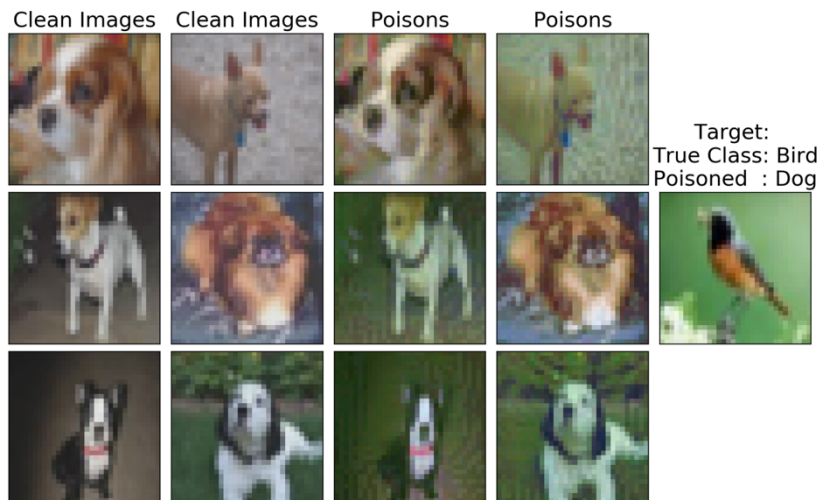
EVALUATION: TRANSFER LEARNING SCENARIO

- MetaPoison vs. Poison Frogs



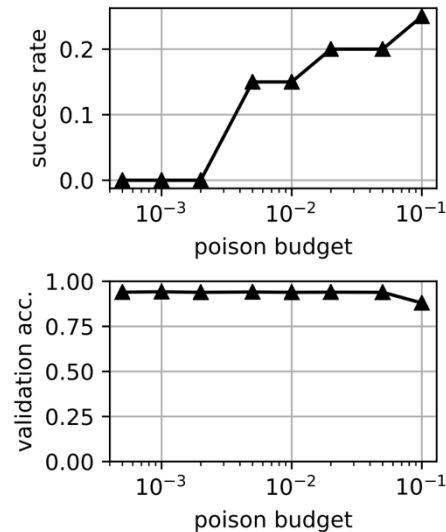
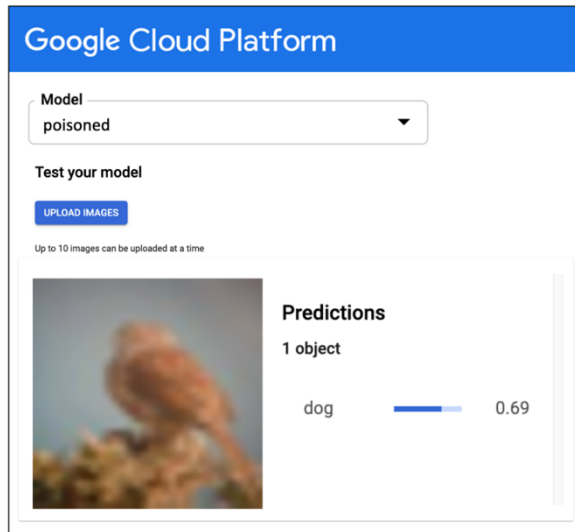
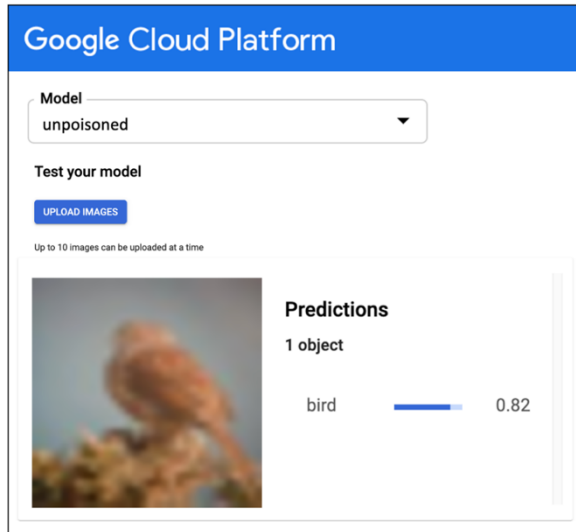
EVALUATION: END-TO-END SCENARIO

- MetaPo



EVALUATION: EXPLOITATION IN REAL-WORLD

- Results



Thank You!

Sanghyun Hong

<https://secure-ai.systems/courses/MLSec/current>



Oregon State
University



TRUE AI
Trustworthy and Responsible AI